

LLM Notes

2026 Spring

Day 2: 权重初始化与 Dropout

deeplearning 基础

主讲: Aaron

Fudan University

今日摘要

今天浅述权重初始化和 Dropout 技术, 这两者学起来都不困难, 这几天比较忙, 我本身也是按照薇薇姐的分享路径来作为笔记轨迹的参考, 刚好来一些轻松的内容

2.1 权重初始化

神经网络 or 大语言模型的本质是优化一个高度复杂的函数, 而权重初始化就是我们优化的起点, 很显然在优化问题中, 一个合理的起点有助于我们高效得找到优秀的解。

我们介绍两种主流的初始化方式: Xavier 与 He(Kaiming)

定义

定义 2.1.1 (Xavier initialization). 设某一层的输入维度为 n_{in} , 输出维度为 n_{out} , 对于权重矩阵 W , Xavier 初始化通常有两种形式:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

or

$$W_{ij} \sim U\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

偏置项 b 一般初始化为 0。

分析

该方法是针对 Sigmoid、Tanh 等饱和激活函数的权重初始化方案，核心目标是保证前向传播、反向传播每层输入输出方差一致。

Xavier 初始化的思想是让信号在网络层与层之间传播时，既不会因为权重过大而导致激活值或梯度爆炸，也不会因为权重过小而导致激活值或梯度逐层衰减。对于线性变换

$$z_j = \sum_{i=1}^{n_{in}} W_{ji} x_i, \quad j = 1, 2, \dots, n_{out}$$

如果假设输入 x_i 和权重 W_{ji} 相互独立且均值为 0，则输出方差大致满足

$$\text{Var}(z_j) \approx n_{in} \text{Var}(W_{ji}) \text{Var}(x_i).$$

而反向传播的梯度方差大致满足：

$$\text{Var}(\partial L / \partial x_i) \approx \sum_{j=1}^{n_{out}} \text{Var}(W_{ji}) \text{Var}(\partial L / \partial z_j) = n_{out} \text{Var}(W_{ji}) \text{Var}(\partial L / \partial z_j)$$

因此初始化中使用 $1/n_{in}$ 确保了正向传播的稳定， $1/n_{out}$ 确保了反向传播的稳定

Xavier 初始化, 使用调和平均

$$\text{Var}(W) = \frac{2}{n_{in} + n_{out}},$$

相当于同时考虑前向传播中的输入维度 n_{in} 和反向传播中的输出维度 n_{out} ，从而在前向激活和反向梯度之间取得平衡。

备注

现状：在现代大语言模型中，Sigmoid / Tanh 几乎不使用，取而代之的是 ReLU 家族（GELU、Swish / SiLU）及其门控变体。因此，Xavier 初始化某种意义上已被时代淘汰。

定义

定义 2.1.2 (He initialization). 也称为 Kaiming initialization，主要针对使用 ReLU 及其变体激活函数的深层神经网络。

设某一层的输入维度为 n_{in} , 输出维度为 n_{out} 。对于权重矩阵 W , He 初始化常用形式为:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

or

$$W_{ij} \sim U\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right).$$

其中, n_{in} 表示当前层每个神经元接收的输入数量。偏置项 b 通常初始化为 0。

分析

He 初始化是对 Xavier 初始化在 ReLU 网络中的改进。ReLU 激活函数定义为

$$\text{ReLU}(x) = \max(0, x).$$

当输入近似关于 0 对称时, ReLU 会将大约一半的负值置为 0, 这会使激活值的方差大约减半。

如果仍然使用 Xavier 初始化, 深层 ReLU 网络中的信号方差可能会逐层变小, 从而导致梯度消失或训练困难。He 初始化通过设置

$$\text{Var}(W) = \frac{2}{n_{\text{in}}},$$

在权重方差中引入额外的系数 2, 用来补偿 ReLU 截断负半轴造成的方差损失。

备注

正态分布与均匀分布权重初始化有两个版本, 两者的均值方差是一致的, 可能在少数情况下均匀分布有一定优势 (其有明确的上下阈值)。

如果严格推导前向和反向, kaiming 初始化也会有 $2/n_{\text{in}}$, $2/n_{\text{out}}$ 两个版本, 但是由于现代神经网络架构中残差连接、自适应优化器的存在, 激活值的稳定性比梯度稳定性更重要一些, 因此一般只关心 $2/n_{\text{in}}$ 。

2.2 Dropout

Dropout 在训练过程中以一定概率随机将神经元的输出置为 0, 防止模型过拟合, 使得模型更分散得学习。

定义

定义 2.2.1 (Dropout). 设某一层的输出为 $\mathbf{h} \in \mathbb{R}^d$, Dropout 首先采样一个与 $\mathbf{h}$ 同形状的掩码向量 $\mathbf{m} \in \{0, 1\}^d$, 其中每个分量独立地以概率 p 取 0, 以概率 $1 - p$ 取 1:

$$\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h}, \quad m_i \sim \text{Bernoulli}(1 - p),$$

参数 p 称为**丢弃率**, 通常取 $0.1 \sim 0.5$ 之间。

分析

Dropout 的本质是一种**模型集成**的近似。每一次前向传播都相当于从原始网络中随机采样一个子网络进行训练, 而最终预测时所有子网络共同作用。数学上, 训练时 Dropout 的输出期望为

$$\mathbb{E}[\tilde{\mathbf{h}}_i] = (1 - p) \cdot h_i.$$

为了保证训练和测试时的输出量级一致, 测试阶段需要对权重或输出进行缩放:

$$\mathbf{h}_{\text{test}} = (1 - p) \cdot \mathbf{h},$$

或者在训练阶段将输出除以 $1 - p$, 即所谓的 **inverted dropout**。

备注

Dropout 与 Batch Normalization 的兼容性问题: 在使用 Dropout 时, 若网络同时包含 Batch Normalization, 通常建议将 Dropout 置于 BN 层之后。此外, 由于 BN 本身已具有一定正则化效果, 二者同时使用时需谨慎调节丢弃率, 避免过度正则化导致欠拟合。

2.2.1 在现代大语言模型中的使用现状

在传统卷积网络和全连接网络中, Dropout 曾是不可或缺的正则化工具。然而, 随着大语言模型的兴起, Dropout 的使用发生了显著变化。

分析

预训练阶段：在现代大语言模型（如 GPT 系列、Llama、Qwen 等）的预训练中，Dropout 通常被设为 **0 或极小值（如 0.0~0.1）**。原因如下：

1. **数据规模足够大：** LLM 的预训练数据规模通常在数万亿 token 级别，过拟合风险远低于小数据集场景；
2. **训练效率优先：** Dropout 引入的随机性会降低训练的收敛速度和吞吐量，影响大规模并行训练的效率；
3. **已有充分的正则化手段：** LayerNorm / RMSNorm、权重衰减（Weight Decay）、以及海量数据本身已能有效控制过拟合；
4. **模型容量极大：** 数十亿甚至万亿参数的网络本身具有较强的泛化能力，Dropout 带来的正则化收益相对有限。

备注

微调阶段：在下游任务的微调中，尤其是在**小样本或参数高效微调（如 LoRA）**场景下，适当引入 Dropout（丢弃率通常设为 0.05 ~ 0.1）仍能带来一定的正则化收益，有助于防止模型在有限的任务数据上过拟合。不过总体而言，Dropout 在 LLM 生态中的地位已远不如传统深度学习中那般核心。